install_cli_agent_plugins.sh

Revision: 2 **Last modified:** 2026-07-14T22:05:00Z **Authority:** constitution §11.4.140 (action-prefix system) + §11.4.164 (post-update auto-propagation) + §11.4.75 (mechanical enforcement) + §11.4.201 (a guard must assert the real condition) **Maintainer:** constitution submodule (inherited by reference per §11.4.177 / §11.4.28(B)) **Scope:** §11.4.18 companion doc for `constitution/scripts/install_cli_agent_plugins.sh`

Overview

`install_cli_agent_plugins.sh` is the **one idempotent command** that makes the constitution's CLI-agent action directives and Agent Skills load out of the box in a consuming project. It is the script the §11.4.164 post-update hook calls on every constitution pull (STEP 4b), and the script each skill's own `register.sh` delegates to.

It does five things, in order:

1. **Regenerate** — runs `generate_agent_prefix_commands.sh`, so every action in `actions/registry.yaml` has a current slash command for every supported agent (Claude Code plugin `helix`, Gemini CLI, Qwen Code, Codex CLI).
2. **Wire the other agents' commands** (`link_agent_commands()`) — symlinks the generated files into each agent's project-level command directory: `.gemini/commands/`, `.qwen/commands/`, `prompts/` (Codex — the destination declared by the repo's own `actions/generated/README.md`). BY REFERENCE, never a copy (§11.4.28 / §11.4.80). It **never clobbers a real file** — it replaces only its own symlink or fills a free slot, and WARNs otherwise.
3. **Wire the prompt hook** (`wire_prompt_hook()`) — chains `install_action_prefix.sh`, which registers the `UserPromptSubmit` expansion hook in `.claude/settings.json`. That installer stays the hook's **single owner** (the `helix` plugin ships no hook — two would double-expand every prompt), and it is genuinely idempotent, so chaining it is safe.

4. **Link skills** — symlinks every constitution skill that has a `SKILL.md` into `<project>/.claude/skills/<name>`, the native project-scoped Agent-Skills discovery path. Again by reference, never a copy.
5. **Install the plugin** — `claude plugin marketplace add <const_root>` + `claude plugin install helix@helix-constitution`, then **verifies the end state with `claude plugin list`**. It does not trust either command's exit code or success message (§11.4.6 / §11.4.200).

System-level narrative: `docs/actions/REFERENCE.md` and `docs/actions/ADMIN_MANUAL.md`.

Prerequisites

- `bash` (`set -euo pipefail`, arrays), `ln`, `find`.
- `scripts/generate_agent_prefix_commands.sh` and `scripts/install_action_prefix.sh` present in the constitution (a missing one is WARNed, never faked).
- The `claude` CLI is **optional**. When absent, the script says so and prints the exact one-time commands rather than claiming success; every other step still completes, and Gemini / Qwen / Codex do not need it at all.
- Indirectly (through the generator): `python3` + `PyYAML` preferred, with an awk fallback for the registry shape this project emits.

Usage examples

```
# Full wire-up from the consuming project root (the normal case)
bash constitution/scripts/install_cli_agent_plugins.sh

# Explicit project root
bash constitution/scripts/install_cli_agent_plugins.sh /path/to/project

# Report state (constitution side AND project side), change nothing
bash constitution/scripts/install_cli_agent_plugins.sh --check

# Only (re-)link the skills
bash constitution/scripts/install_cli_agent_plugins.sh --skills-only

# Link exactly one skill (what skills/<name>/register.sh calls)
bash constitution/scripts/install_cli_agent_plugins.sh --skill action-prefix-system

# On a host with no `claude` CLI
bash constitution/scripts/install_cli_agent_plugins.sh --no-plugin
```

Real full-install output:

```
[install-plugins] regenerated slash commands from registry (10 Claude Code
command file(s))
[install-plugins] wired 10 gemini command(s) → .gemini/commands/
[install-plugins] wired 10 qwen command(s) → .qwen/commands/
[install-plugins] wired 10 codex command(s) → prompts/
[install-plugins] wired the UserPromptSubmit prefix hook (free-form
'NAME :: task' forms)
[install-plugins] linked 3 skill(s) into .claude/skills/ (4 legacy dir(s)
without SKILL.md skipped)
[install-plugins] plugin 'helix@helix-constitution' is INSTALLED (verified
via `claude plugin list`)
[install-plugins] done — action directives + skills are wired for this
project
```

Real `--check` output on a fully-wired tree:

```
[install-plugins] constitution side: marketplace + plugin manifests
present; 10 command(s); 3 skill(s)
[install-plugins] project side: 3 skill symlink(s) in .claude/skills/
[install-plugins] project side: UserPromptSubmit prefix hook is registered
[install-plugins] project side: plugin 'helix' is installed + enabled
[install-plugins] VERDICT: WIRED
```

Edge cases

Case	Behaviour
`claude` CLI not on `PATH` (install)	WARNs, prints the two one-time commands, **does not** fail the whole install (`install_plugin || true`). Honest, never faked.
`claude` CLI not on `PATH` (`--check`)	Plugin state reported as **UNKNOWN** — not installed, not missing. A guard must assert the real condition or say honestly that it could not (§11.4.201).
`claude plugin list` does not show `helix` after install	WARNs explicitly ("NOT claiming success") and prints the commands to run manually and read the error.
A **real file** (not a symlink) already sits at `.gemini/commands/<name>.toml` (or the qwen/codex equivalent)	Left untouched, with a WARN. The operator's hand-written command wins; the installer never clobbers it.
An existing **symlink** at that path	Replaced (`rm -f` then `ln -s`) — re-linking is idempotent.
`actions/generated/` missing	WARN "other agents NOT wired"; `rc=1` ; the rest continues.
A skill dir has **no** `SKILL.md`	**Skipped** — not discoverable as an Agent Skill. The skipped count is reported (4 legacy dirs today: `media-validator` , `scheduled-work-queue` , `session-sync` , `multitrack`).
`.claude/skills/<name>` exists as a **real directory**	Left untouched, with a WARN. Remove it and re-run.

Case	Behaviour
Repeated runs	Fully idempotent. Identical generated files, identical symlinks; marketplace/plugin add are no-ops; the `UserPromptSubmit` hook stays registered **exactly once** (verified: `grep -c action_prefix_expand .claude/settings.json` → `1` after three consecutive installs — two entries would double-expand every prompt).
Generator or `install_action_prefix.sh` missing / failing	WARN; `rc=1`; the script continues with the remaining steps and reports "completed WITH problems".
`--skill NAME` where `NAME` does not exist	`linked 0 skill(s)`; not an error.

Honest boundary (§11.4.6) — what this script does *not* do:

- It does **not** own the `UserPromptSubmit` hook — it *chains* the installer that does (`install_action_prefix.sh`). The split is deliberate: the `helix` plugin ships commands only, so a prompt can never be double-expanded.
- `--skills-only` / `--skill NAME` skip generation, the per-agent command links, the prompt hook, and the plugin install — they only link skills.
- The script's own in-source header block still describes the earlier three-step behaviour (its Purpose / Outputs / Side-effects sections predate `link_agent_commands()`, `wire_prompt_hook()`, and the project-side `--check`). This companion doc reflects the **code**, which was read line by line.

Internal behaviour

```
argv → project_root (default $PWD), flags
  │
  ├─ --check ──────→ CONSTITUTION side: marketplace.json + plugin.json +
n_cmds>0 + n_skills>0
  │                  PROJECT side:      symlink count in .claude/skills/
(≥ n_skills)
  │                                     grep action_prefix_expand .claude/
settings.json
  │                                     claude plugin list | grep helix
(UNKNOWN if no CLI)
  │                  VERDICT: WIRED | NOT-WIRED; exit rc
  │
  ├─ regen_commands()        (skipped under --skills-only)
  │     bash scripts/generate_agent_prefix_commands.sh
  │
  ├─ link_agent_commands()   (skipped under --skills-only)
  │     specs = "gemini:.gemini/commands:toml qwen:.qwen/commands:toml
codex:prompts:md"
  │     for each: symlink actions/generated/<agent>/*.<ext> → <project>/
<dst>/
  │               only when the destination is our own symlink or a free
slot
  │
  ├─ wire_prompt_hook()      (skipped under --skills-only)
  │     bash scripts/install_action_prefix.sh "$project_root"
  │
  ├─ link_skills()
  │     for each constitution/skills/*/ with a SKILL.md:
  │         ln -s → <project>/.claude/skills/<name>   (symlink or free
slot only)
  │
  └─ install_plugin()        (skipped under --skills-only or --no-plugin)
        command -v claude || { print one-time commands; return 1 }
        claude plugin marketplace add "$const_root"     || true   ← exit
code NOT trusted
        claude plugin install helix@helix-constitution  || true   ← exit
code NOT trusted
        claude plugin list | grep -q helix                        ← the
END-STATE assertion
```

Constants: `MARKETPLACE_NAME=helix-constitution`, `PLUGIN_NAME=helix`. `const_root` is resolved from the script's own location, so it works from any cwd. Exit code `0` when generation, linking and skill-wiring succeeded; a missing `claude` CLI alone does not fail the run.

Related scripts

Script	Relationship
`generate_agent_prefix_commands.md`	The generator this script invokes in step 1.
`scripts/install_action_prefix.sh`	Owns the `UserPromptSubmit` hook; **chained** by this script in step 3.
`scripts/post_update_hook.sh`	Calls this script (STEP 4b) on every constitution pull when `actions/**`, `plugins/**`, `.claude-plugin/**` or `skills/**` changed.
`scripts/action_prefix_lib.sh`	The engine behind the generator and the hook.
`skills/*/register.sh`	Each delegates to this script with `--skill <name>`.
`scripts/reporting/report_item.sh`	The engine the `BUG` / `TASK` / `ISSUE` directives call once wired.

Last verified

2026-07-14 — read against `scripts/install_cli_agent_plugins.sh` at constitution HEAD (post `link_agent_commands()` / `wire_prompt_hook()` / project-side `--check`); every documented flag, message and edge case confirmed line by line, and the full install + `--check` outputs above captured from real runs (exit 0), including the hook-registered-exactly-once invariant.